API SECURITY RISK ANALYSIS REPORT

Nhlosenhle Mkhonta

Table of Content:

API Tested……2

Methodology…….2

Identified Risks…….2

Findings table……4

Risk Severity Classification……..4

Business Impact…….4

Remediation Recommendations……5

1. API Tested

- **Name**: ReqRes API
- **Base URL**: https://reqres.in
- **Purpose**: Demo API for testing authentication, CRUD operations, and user data handling.

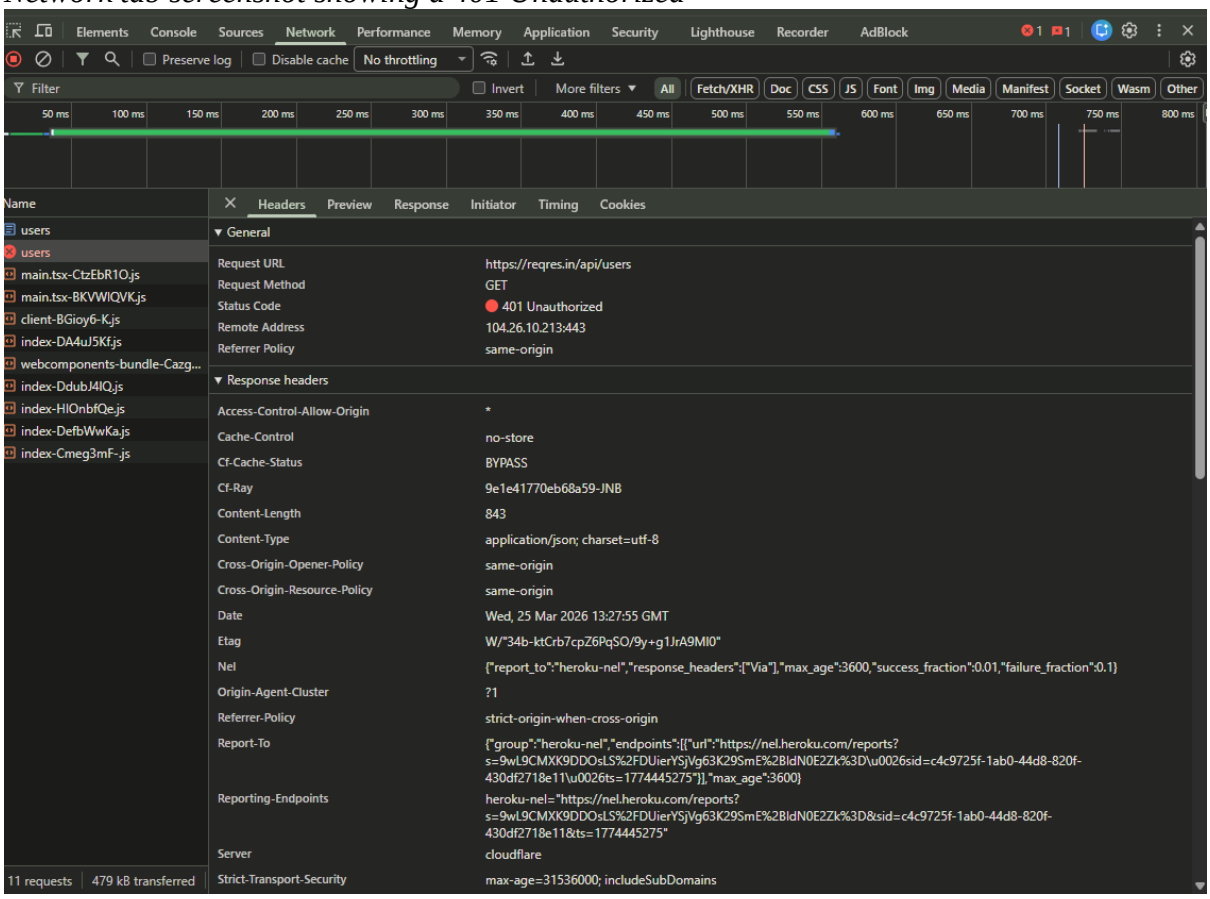
2. Methodology

- **Tools Used**: Postman, Browser DevTools
- **Scope**: Read-only inspection of endpoints, headers, and responses
- **Approach**:

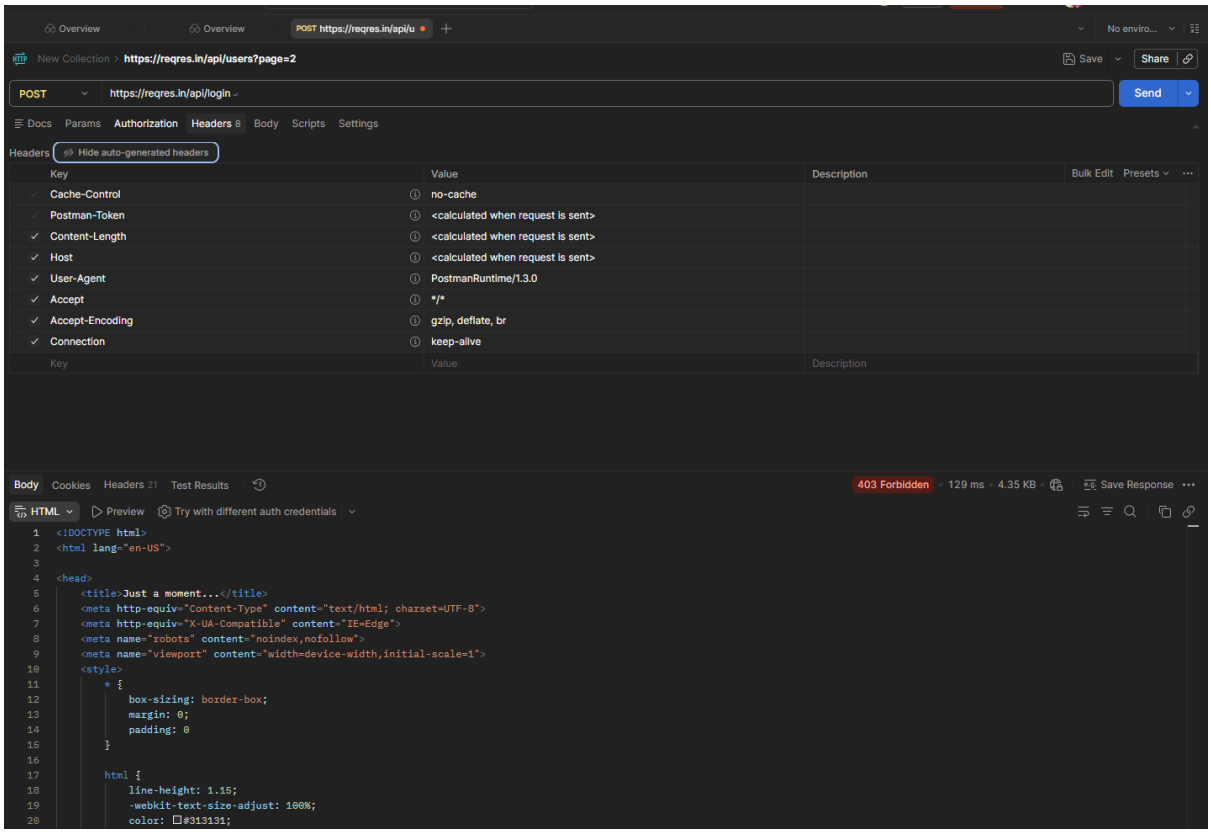
- Reviewed documentation
- Sent test requests (GET /users, POST /login)
- Inspected authentication requirements, headers, and response payloads
- Classified risks by severity and business impact

3. Identified Risks

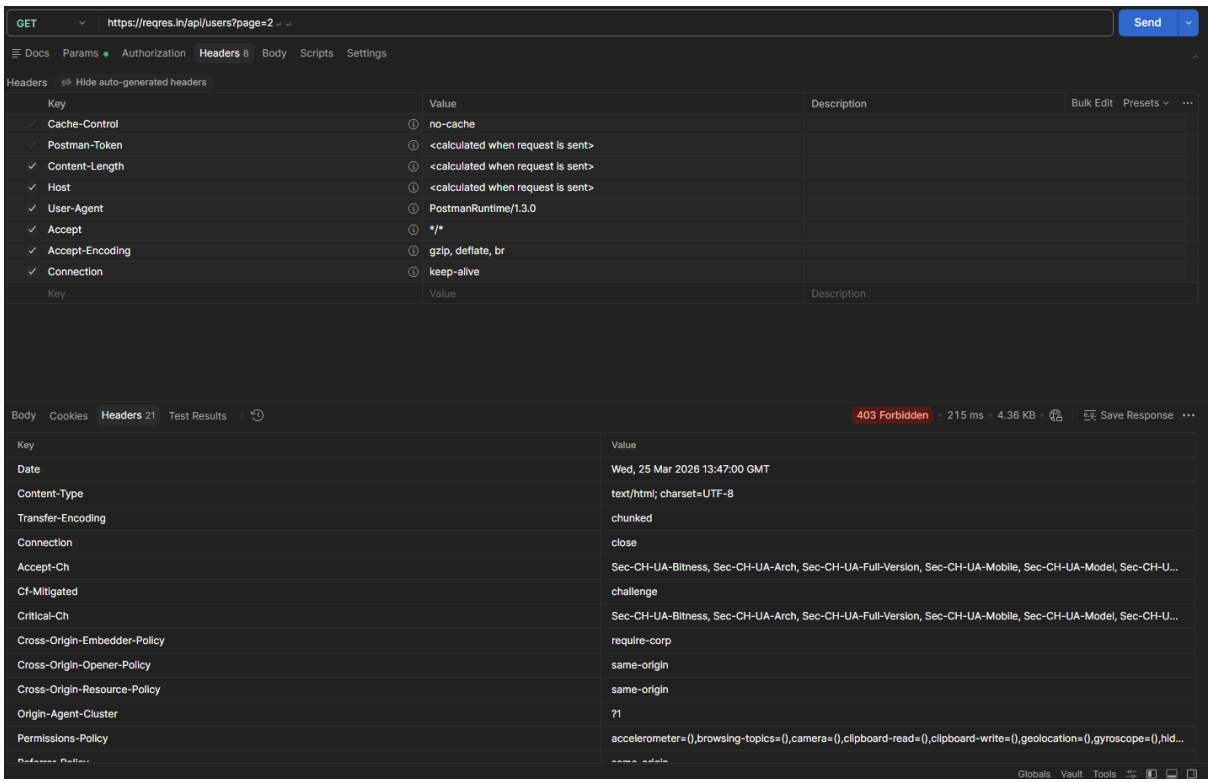
Network tab screenshot showing a 401 Unauthorized



successful POST request screenshot



successful GET request screenshot



Findings table:

Risk	Evidence	Severity	Business Impact	Remediation
Unauthorized Access	401	High	Endpoints reject requests without credentials, but error messages may expose metadata.	Enforce authentication and sanitize error responses.
Forbidden Request	403 Forbidden (Postman GET)	Medium	Access control is enforced, but inconsistent error handling could confuse users.	Standardize error handling and role-based access control.
Weak Login Simulation	403 Forbidden	High	Login flow not properly secured; in real SaaS, this could allow token replay or denial of service.	Implement secure token-based authentication with expiry and refresh.
Open Endpoint	200 OK	Medium	User data accessible without authentication; in real SaaS, this could expose sensitive information.	Require authentication for user data endpoints.

4. Risk Severity Classification

- **High**: Weak authentication, missing rate limiting
- **Medium**: Open endpoints, lack of input validation
- **Low**: Excessive data exposure (demo data only)

5. Business Impact

If these risks existed in a real SaaS platform, they could lead to serious consequences. Unauthorized access and weak authentication would expose sensitive customer data, resulting in privacy violations and regulatory fines. Inconsistent error handling could leak system details, giving attackers clues about the backend. Open endpoints could allow attackers to scrape user data. Missing rate limiting would enable brute-force or denial-of-service attacks, leading to downtime and reputational damage. Together, these issues could result in financial losses, compliance failures, and erosion of customer trust.

6. Remediation Recommendations

- Use JWT tokens with expiry and refresh mechanisms.
- Apply rate limiting (e.g., 100 requests/min per IP).
- Validate and sanitize all inputs.
- Return minimal data fields in responses.
- Standardize error messages to avoid leaking system details.
- Implement monitoring and logging for unusual API usage.
- Require authentication for all sensitive endpoints.